SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

Chapter 2 — Data Sourcing
From data types to APIs, scraping and primary data collection

Lecturer: Xieer Dai (David)

Shenzhen University · Weeks 3–14 · Thursday, periods 11–12



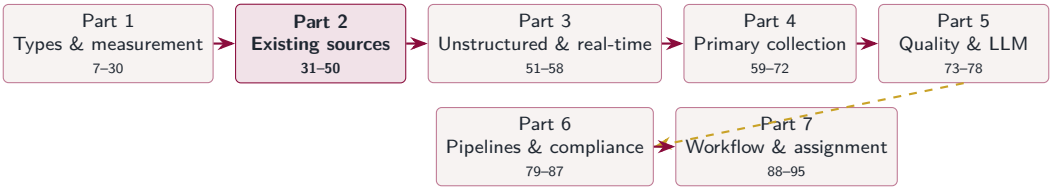
Learning Objectives

After this lecture you should be able to:

1. distinguish data types, structures (cross-section / time-series / panel) and levels of measurement, and choose methods accordingly;
2. evaluate a data source along coverage, quality, cost, compliance and reproducibility;
3. explain the difference and use cases of in-house proprietary, open, third-party (3P) data, alternative data and synthetic data;
4. acquire data properly with APIs (REST / auth / pagination / rate limits / caching) and web scraping, while respecting legal and ethical boundaries;
5. design surveys, sampling and A/B experiments for primary collection, handling nonresponse bias and labelling consistency;
6. deliver a reproducible data-source inventory + acquisition script, declaring any AI tools you used.

Source: Course team material; data-source and compliance points current to 2026 – sources are footnoted on each slide.

Roadmap (95 slides)



Spine: where does data come from → how do we get it → is it usable → is it compliant

Code slides (9): API calls, scraping, PDF extraction, LLM extraction, data health check, minimal pipeline

Sample slides (10): each data type with a sample and visualisation (incl. line-by-line commented Python)

From L1 to L2

Questions left by L1

- How exactly is the first step of the data pipeline — collection — done?
- What are open data, third-party data, APIs and scraping each good for?
- Where do data-quality and compliance risks come from, and how do we control them early?

L2's answer

- Decompose "data sources" into 3P + APIs + alternative data + synthetic data;
- give working acquisition code and compliance red lines;
- deliver a data-source inventory and reproducible acquisition scripts.



Deliverables

1) Data-source inventory

Source, fields, time coverage, update frequency, cost, licence, quality score, owner (template on slide 89)

2) Acquisition script

Re-runnable API/scraping code: authentication, pagination, retries, caching, persistence and logging (slides 45–48, 52–54)

3) Compliance checklist

Authorisation, personal information, request rate, attribution, redistribution, cross-border transfer (slides 50, 86–87)

Assignment 1 (set in Week 3 / due in Week 4)

Pick a dataset → complete the data-source inventory + acquisition script + data health-check report → attach an AI-use declaration (details on slide 90)

Source: Inventory and script templates come with the handout; all example data sources are public or available on application.

Map of the Lecture

Part	Slides	Key questions
Data types & measurement	7–30	What data do I have? What is its structure? Is the measurement reliable?
Existing data sources	31–50	What are the entry barriers and costs of in-house / open / third-party / platform-API data?
Unstructured & real-time	51–58	How do PDFs, web tables and text become structured data? When is real-time needed?
Primary data collection	59–72	How do surveys, sampling, experiments, interviews and card sorting actually work?
Quality control & LLM	73–78	How do we ensure labelling consistency? Can LLM-assisted collection be trusted?
Pipelines, governance, compliance	79–87	How do we wire sources into a reproducible, auditable data pipeline?
Workflow & assignment	88–95	Inventory template, code cheat sheet, assignment requirements and grading.

From the World to a Variable



Data does not "exist in nature": every operationalisation step injects assumptions. In a paper, state which variable proxies the concept, why that is defensible, and where the limits lie.

Measurement is a design decision: the same concept can be operationalised in several ways, each with different validity and bias.

Three reminders

(1) An indicator $\neq$ the concept (GDP is not "development", click-through rate is not "interest"); (2) a definition change breaks a time series; (3) documenting the operationalisation itself is part of reproducibility.

Levels of Measurement

Level	Meaning	Allowed operations / examples
Nominal	Categories only, no order	counts, contingency tables / industry codes, gender, stock tickers
Ordinal	Ordered, unequal gaps	comparisons, rank correlation / credit ratings AAA–C, Likert scales
Interval	Equal intervals, arbitrary zero	addition, subtraction, means / Celsius temperature, years, some scores
Ratio	True zero point	multiplication, division, ratios / income, market cap, trading volume, population

Treating ordinal data as interval and taking means is the most common hidden error (e.g. regressing on rating codes 1–5 as if numeric, without justification).

The level of measurement determines which statistics are meaningful — and which conclusions are defensible.

Numerical Data

Continuous vs discrete

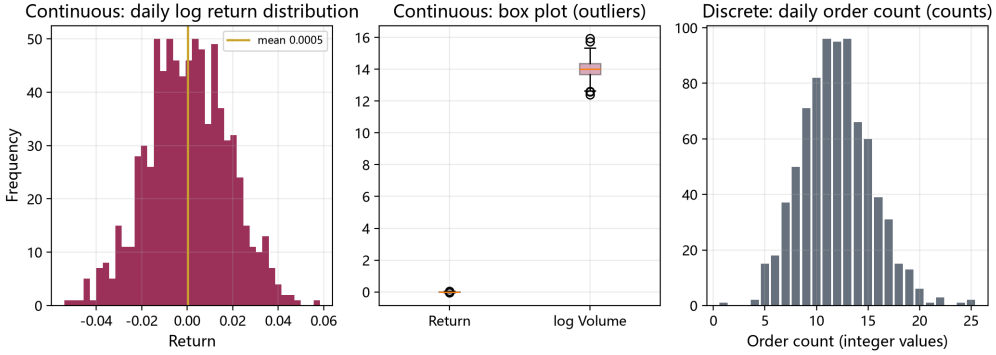
- Continuous: prices, returns, temperature (recording precision decides how discrete they "look");
- Discrete: order counts, clicks, accidents (count data, often needs Poisson-type models);
- Mixed: money is a continuous concept; booked in cents it becomes discrete.

Engineering points

- Units and scales: yuan / 10k yuan / 100m yuan; USD / CNY; always state the arithmetic basis;
- Precision and rounding: floating-point error, rounding policy, use Decimal for money;
- Zero point: ratio scales support growth rates directly; interval scales should not be used for multiples.

Source: Standard measurement-theory statements; engineering points compiled from data-analysis practice.

Numeric: Sample & Visualisation



Sample `numeric_samples.csv` (800 trading days): continuous (daily log returns, volume) and discrete counts (daily order numbers).

Plot by type: histogram for one variable, boxplot for outliers, count bar for discrete values.

Code `01_datatype_numeric.py` · sample `numeric_samples.csv` (course repo website/code and website/data)

Categorical Data

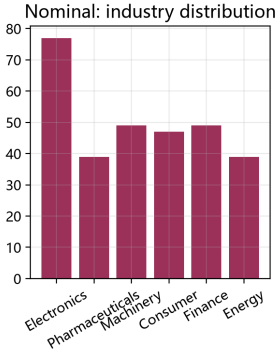
- Nominal (industry, region, brand) vs ordinal (ratings, satisfaction);
- High cardinality: tickers, postcodes, SKUs — one-hot encoding explodes;
- Encoding options: one-hot, ordinal, target encoding (cross-fit to avoid leakage), embeddings;
- Unknown/new categories: production systems must design an "other/unknown" branch.

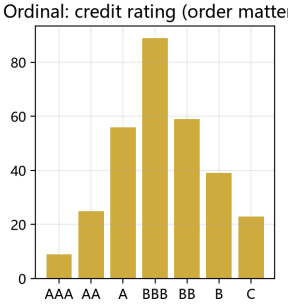
Common traps

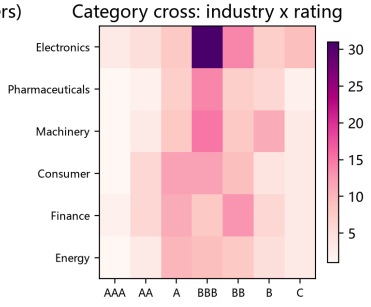
- Treating category codes as numbers: industry code 13 vs 26 has no ordering;
- Category definitions that shift over time (administrative re-zoning, industry classification revisions);
- Missingness itself is information: don't confuse "not answered" with "no".

Chinese-language corpora add extra issues: abbreviations, aliases, traditional characters, typos — entity alignment is often part of the sourcing step.

Categorical: Sample & Visualisation







Sample `categorical_samples.csv` (300 companies): nominal (industry, 6 classes) and ordinal (credit rating AAA–C, 7 levels).

Nominal: no order. Ordinal: order matters, so keep the order in plots and models.

Code `02_datatype_categorical.py` · sample `categorical_samples.csv`

Structured / Semi-structured / Unstructured

Structured: a 2D table of fixed-length fields (CSV)

order_id	date	amount	paid
SO2026001	2026-03-02	1280.5	True
SO2026002	2026-03-02	640.0	True
SO2026003	2026-03-03	3980.75	False
SO2026004	2026-03-03	215.2	True

Semi-structured: nested JSON (schema can change)


```
{"order_id": "SO2026001",
"buyer": {"name": "Lin", "
vip": true}, "items": [{"s
ku": "A-1", "qty": 2}, {"s
ku": "B-7", "qty": 1}]}
```


Fields may be omitted or added: parsing must tolerate this

Unstructured: text / audio / image

Revenue in the Southea
st Asian market grew 3
0% YoY this quarter, m
ainly driven by cross-
border e-commerce chan
nels; management expec
ts the gross margin to
stay stable next quar
ter, but FX swings and
rising ocean freight
add cost pressure.



Text / audio / image: no fields to enumerate

Structured: fixed rows and columns, ready for databases | Semi-structured: JSON/logs with optional, nested fields — parsing must tolerate change | Unstructured: text/images/audio — convert to vectors or features first.

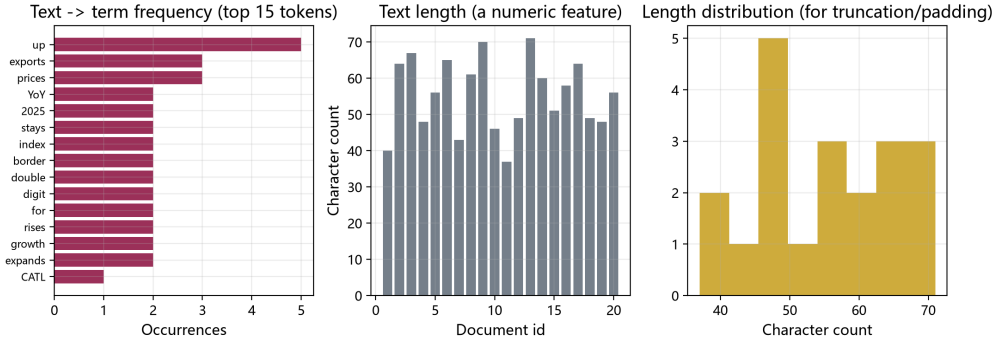
Code `10_datatype_structured_unstructured.py` · samples `structured_orders.csv` and two more

Complex & High-dimensional Types

Type	Typical sources	Processing entry point (later lectures)
Text	news, announcements, reports, social media, support tickets	tokenisation / embeddings / RAG (L10–L11)
Image	satellite, store cameras, product photos, receipts	CNNs / pretrained backbones (L9)
Audio	call recordings, service quality checks, voice assistants	speech recognition + text analysis (L9–L10)
Video	short video, livestreams, surveillance	frame sampling / multimodal models (L9)
Graph / network	transfers, supply chains, social, citations	graph algorithms & GNNs (L9)
Spatio-temporal tracks	GPS, AIS, card swipes, food delivery	panels & spatial econometrics (L14)

The data type constrains the method: first ask "can this become numbers/vectors?", then choose the modelling route — not the other way round.

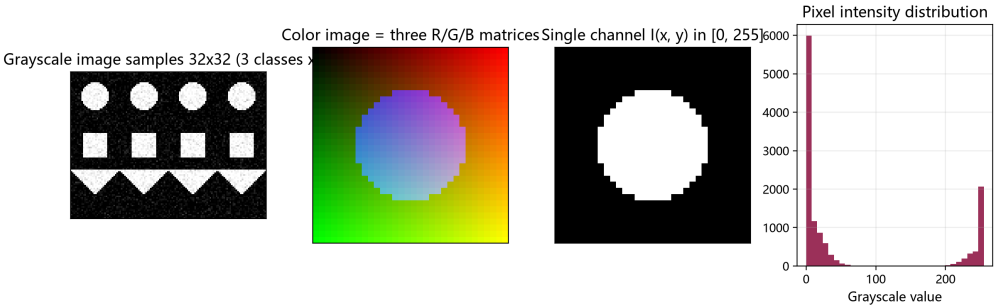
Text Data: Sample & Visualisation



Sample `text_samples.csv` (20 finance headlines): tokenise, count word frequencies and document lengths; the length distribution drives truncation and padding choices.

Code `website/code/03_datatype_text.py` · entry point: tokenisation / embeddings / RAG (L10–L11)

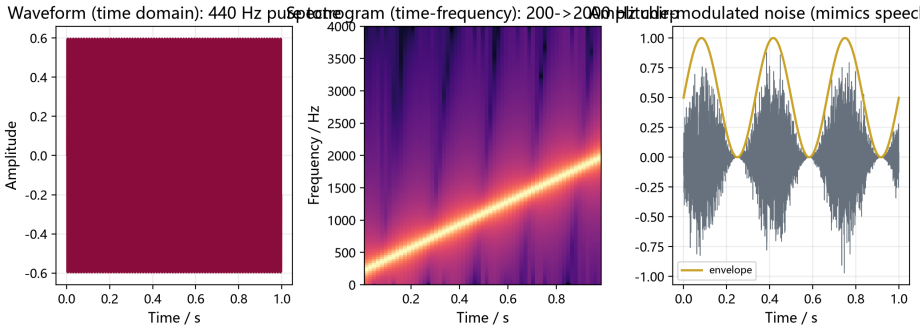
Image Data: Sample & Visualisation



Sample `image_samples.npy` (3 classes $\times$ 4 images, 32$\times$32 grayscale): an image is a matrix; a colour image stacks three matrices.

Code `website/code/04_datatype_image.py` · entry point: CNNs and pretrained backbones (L9)

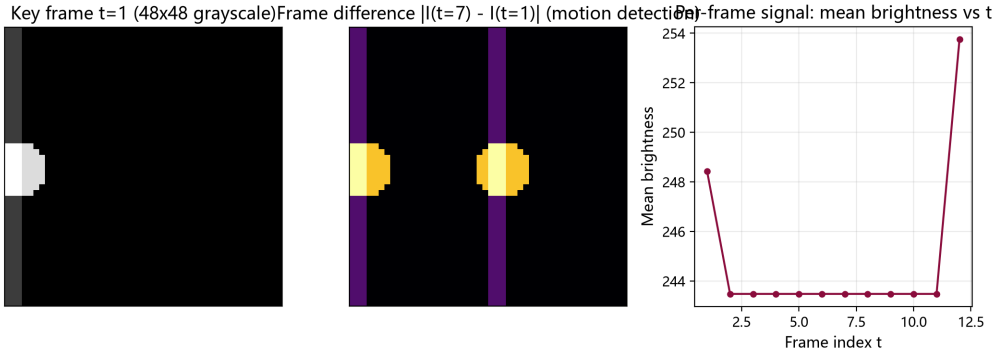
Audio Data: Sample & Visualisation



Sample `audio_*.wav` (8 kHz, 1 s): the sampling rate bounds the representable frequencies; the spectrogram gives a time–frequency view.

Code `website/code/05_datatype_audio.py` · entry point: speech recognition + text analysis (L9–L10)

Video Data: Sample & Visualisation

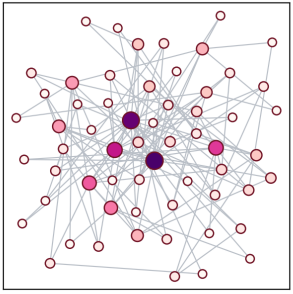


Sample `video_frames.npy` (12 frames, 48×48): video = frame sequence + time axis; most tasks only need frame sampling or frame differencing.

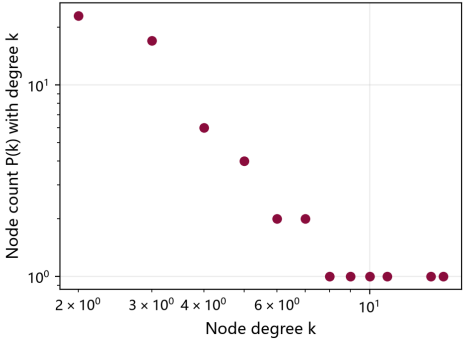
Code `website/code/06_datatype_video.py` · entry point: frame sampling and multimodal models (L9)

Graph & Network Data: Sample & Visualisation

Network graph (60 nodes / 117 edges, BA scale-free)



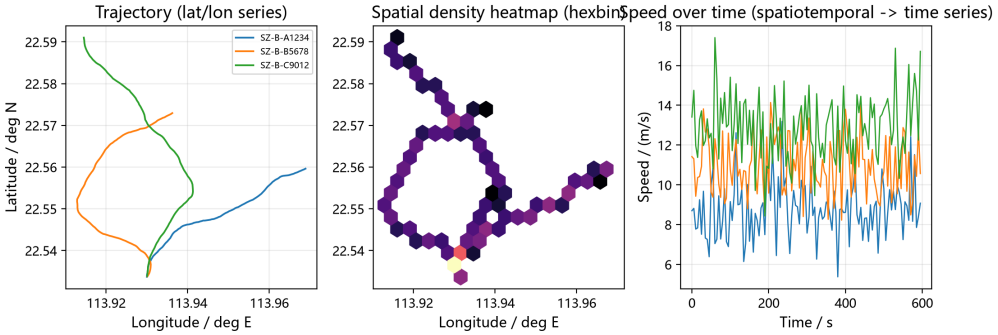
Degree distribution (log-log: power-law feature)



Samples `graph_edges.csv` and `graph_nodes.csv` (60 nodes, BA scale-free): the degree distribution is close to a power law.

Code `07_datatype_graph.py` · entry point: graph algorithms and GNNs (L9)

Spatio-temporal Data: Sample & Visualisation



Sample `spatiotemporal_tracks.csv`: 3 trajectories $\times$ 120 points of latitude/longitude time series.

Code `08_datatype_spatiotemporal.py` · entry point: spatial econometrics and panels (L14)

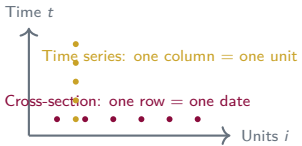
Data Structure A: Cross-section and Time-series Data

Cross-section

- Many units at the same point in time: provincial GDP in 2025, company market caps;
- good for cross-sectional comparison, distribution and inequality analysis; limitation: cannot separate "unit differences" from "time effects".

Time-series

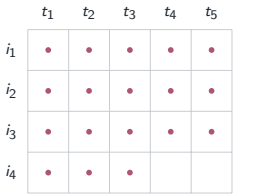
- Many time points for the same unit: CSI 300 daily closes, monthly CPI;
- good for trends, seasonality, volatility and forecasting; limitation: autocorrelation, non-stationarity and structural breaks.



Source: Standard cross-section / time-series / panel taxonomy.

Data Structure B: Panel Data

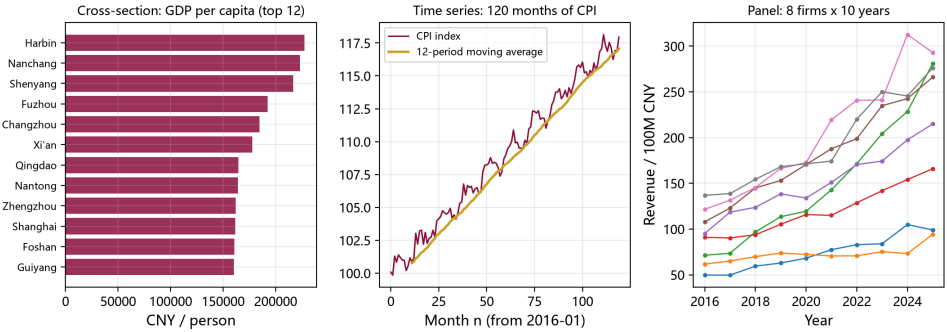
- Definition: a 2-D table of units $i \times$ time t (balanced/unbalanced panel);
- Strength: uses variation between and within units; controls unobserved unit heterogeneity;
- Typical sources: listed-firm annual reports (firm panels), customs–firm matches (L14), household panels (CFPS);
- Key handling: unify entity codes, sample entry/exit, definition changes, missing patterns.



blank = unit exits or data missing

Source: Standard panel-data structure and strengths; CFPS and similar surveys are common micro-panel sources.

Cross-section / Time-series / Panel: Visual Comparison



Cross-section: distribution and ranking (comparing units at one date).

Time series: trends, seasonality and breaks (one unit over time).

Panel: unit differences and common trends (both used together).

Code `09_datastructure_cross_time_panel.py` · samples `structure_*.csv` (one each: cross-section / time series / panel)

Data Structure C: Cross-section Dependence, Spatial and Social Networks

Cross-section dependence

Units at the same date affect each other: price wars within an industry, policy spillovers within a region. Ignoring dependence understates standard errors.

Spatial data

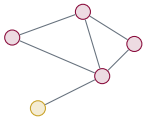
Geographic adjacency or distance weight matrix W; spatial autocorrelation (Moran's I); spatial panel models.

Independent panel

Units are approximately independent; standard panel methods apply.

Network data

Node–edge structures: transaction networks, supply chains, social and citation networks; degree / centrality / communities.



a "high centrality" node

Source: Standard spatial-econometrics and network-analysis concepts; social-network data sources on slides 43 and 56.

Structured, Semi-structured and Unstructured

Class	Characteristics and examples	Acquisition and processing
Structured	Fixed rows/columns: market data, financial-statement fields, transaction ledgers, government statistics	direct database/API access, explicit schema
Semi-structured	Self-describing but variable: JSON/XML, logs, HTML, e-mail headers	parser + field extraction + normalisation
Unstructured	No fixed form: news, announcement PDFs, audio, images, video	OCR/ASR/embeddings/LLM extraction (needs verification)

In reality about 80% of new data is unstructured or semi-structured; "structuring" it is the bulk of the sourcing workload.

Market data is structured; behaviour, language and images are not — the sourcing pipeline is where the mess is cleaned up.

Source: The structured/unstructured taxonomy follows the original deck (market-data and social-media examples).

Organising Data for Analysis

Wide

One row = one unit, one column per metric: suited to modelling and pre-modelling feature matrices.

Aggregation levels

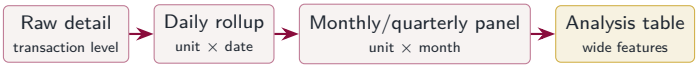
detail → daily rollup → monthly rollup → unit-level; every aggregation discards information, so record the definitions.

Long (tidy)

One row = one unit–variable–value triple: suited to unified storage and plotting across periods and metrics.

Additivity

Amounts/quantities add up; ratios, prices and indices cannot be added directly — weight them (e.g. weighted average returns).



Aggregation Traps

- Ecological fallacy: group-level correlation $\neq$ individual correlation (province-level education–income links do not hold at person level);
- Simpson's paradox: aggregate trend runs opposite to subgroup trends (treated formally in L12 causal inference);
- Mean trap: long-tailed data mislead if described by the mean — use medians/quantiles;
- Denominator drift: numerator/denominator definitions shift (e.g. which population base does "per capita" use).

Practical advice

- Keep the raw granularity; aggregations are derived views only;
- document each aggregation: group keys, time window, weights, missing handling;
- use distribution plots to check whether the story changed after aggregation.

Source: Ecological fallacy and Simpson's paradox are classic statistics concepts; read together with L4 visualisation and L12 causal inference.

Sampling & Population

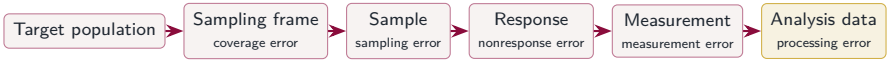
- Target population (whom we want to infer about) vs accessible population (whom we can actually reach);
- Sampling frame: where does the list come from? Who is missing? (elderly without phones, unregistered firms, people offline)
- Representativeness: sample vs population distributions (compare age/region/size structure);
- Sample size: determined by effect size, variance, significance level and power — not "the bigger the better".

Sampling vs census

- Census: full coverage, but costly, slow, few questions;
- Sample: fast, cheap, more detail possible — but must handle sampling and nonresponse error.

Source: Standard sampling theory; China's population census (NBS) is a typical census source — see slide 66.

The Error Chain



Every link can introduce systematic bias; when reporting, state the dominant error sources and their magnitude, not just a point estimate.

Total survey error framework: coverage, sampling, nonresponse, measurement and processing error all matter — and they interact.

Links to later lectures

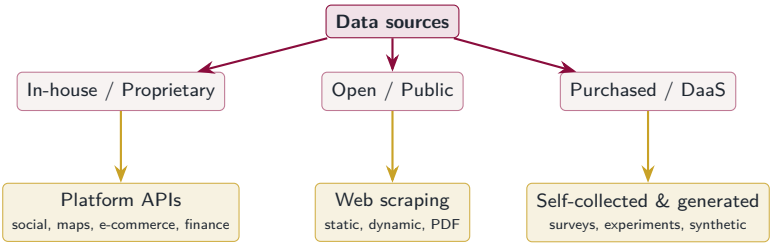
error → interval estimates and hypothesis tests (L4); nonresponse and selection → external validity of causal identification (L12).

Summary: Ten Questions to Ask of Any Dataset

1. Who collected it, and for what purpose? (interested parties)
2. Who is covered, who is missing? (coverage error)
3. Time coverage and frequency? Have definitions changed?
4. Units, currency, nominal/real, tax basis?
5. How does missingness arise? Missing rate and pattern?
6. How are variables measured? Reliability and validity evidence?
7. Can it legally be used, redistributed, transferred across borders?
8. Update frequency and lag? Can acquisition be automated?
9. Is it reproducible (versions, snapshots, scripts)?
10. Whom could its use harm? (ethics and compliance)

Pin this checklist at the start of a project: it shapes research quality more than any model.

The Data Sourcing Landscape



The original deck's 3P: Proprietary, Public, Purchased; the 2026 edition adds three more: platform APIs, alternative data and synthetic data.

Whatever the channel, three questions decide: coverage, quality, and the right to use the data.

Source Evaluation: A Five-Dimension Score Card

Dimension	Questions to ask	Score (1–5)
Coverage	Which units / periods / regions are covered? Who is missing? Gaps in the sampling frame?	
Quality	Are definitions stable? Missing rate? Validation checks? Historical revisions?	
Cost	purchase price + cleaning labour + storage and compute + maintenance	
Compliance	licence scope, personal information, cross-border transfer, redistribution	
Reproducibility	scriptable acquisition? versions / snapshots? interface stability?	

How to use: score the candidate sources first, then weight them (compliance and coverage usually act as veto items), and finally record the result in the data-source inventory.

Cheap but non-compliant is not cheap: it is a liability. Score before you code.

In-house Data (Proprietary)

- **Business systems**: ERP (orders, inventory, cost), CRM (customers, follow-ups), finance;
- **Logs and instrumentation**: page views, clickstreams, app events, API call logs;
- **Transactions and interactions**: payment flows, invoices, support tickets, e-mail and call records;
- **Operations**: scheduling, logistics, warehousing, device sensors (IoT).

Strengths

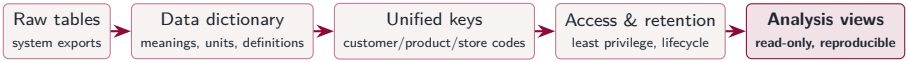
Richest detail, freshest, traceable, directly linkable to business outcomes (a natural place for causal experiments).

Traps

Fragmented definitions (same name, different meaning), short retention, messy permissions, field meanings kept as tribal knowledge, exports become "dead data".

Source: In-house taxonomy compiled from enterprise data-management practice; the 3P frame follows the original "data science needs data" deck.

Turning In-house Data into Usable Data



Four must-do actions

- Build a data dictionary: fields, units, value ranges, owners;
- unify entity keys: customer/product/supplier/store code mapping table;
- version the definitions: record effective dates of metric changes;
- create read-only analysis views: decouple from analysis code, avoid writing to production.

Compliance-related

- Flag and mask personal-information fields separately;
- log export activity (who, when, what);
- auto-purge or archive when the retention period expires.

International Open Data

Source	Content	Access
World Bank	global development indicators (GDP, trade, education, health; 200+ countries, decades)	open API (v2, JSON/XML) + bulk CSV
IMF	international financial statistics (IFS), World Economic Outlook (WEO), FX and balances	data portal + API/SDMX
OECD	member-country socio-economic statistics, structural indicators	data portal + SDMX API
UN Comtrade	bilateral trade flows (by HS code, country, year)	web + API (limited free quota)
Eurostat	EU statistics (labour, prices, trade, regions)	open API (JSON-stat)
FRED	US and global macro time series (800k+ series)	REST API, key required

Source: Open-data portals of each institution (2026): data.worldbank.org, data.imf.org, data.oecd.org, comtrade.un.org, ec.europa.eu/eurostat, fred.stlouisfed.org.

Open Data (International): Research and Web Data

Research and scholarly data

- Zenodo / Figshare: data and code behind papers (citable DOIs);
- ICPSR / UK Data Service: social-science micro-survey data;
- World Bank Microdata Library, AEA Data Repository (paper replication data);
- Data citation convention: author, year, dataset name, version, DOI.

Web and corpus data

- Common Crawl: monthly web snapshots (petabyte scale; you build the parsing pipeline);
- Wikipedia / Wikidata: structured knowledge and multilingual text;
- arXiv / OpenAlex / Crossref: publication metadata and citation networks;
- Read before use: robots, licence (CC variant), redistribution limits.

Research data is "citable and reproducible"; web corpora are "large", at the price of cleaning cost and compliance risk.

Open Data (China): Government Statistics and Regulatory Disclosure

Source	Content	Access
National Bureau of Statistics	monthly/quarterly/annual macro data (GDP, CPI, PPI, investment, retail sales, census)	website queries + data interface (data.stats.gov.cn, unofficial docs)
Local open-data portals	city/province open-data platforms (transport, environment, fiscal, public services)	web portals + some API/CSV
People's Bank of China	aggregate financing, credit balances, interest and exchange rates, payment-system reports	statistical tables on site (Excel/PDF)
General Administration of Customs	import/export by commodity and country (monthly, HS codes)	online customs statistics query platform
Exchanges	quotes, announcements, margin trading, block trades, index constituents	SSE/SZSE/BSE websites + official data interfaces (mostly licensed)

Source: Institution websites (2026): stats.gov.cn, pbc.gov.cn, customs.gov.cn, sse.com.cn, szse.cn; some interfaces are undocumented — assess stability and compliance yourself.

Open Data (China): Market Data and Research Databases

Publicly available market information

- **cninfo**: listed-company announcements, annual reports, prospectuses (full PDFs);
- **SSE / SZSE**: trading disclosures, margin data, top-mover lists;
- **National Enterprise Credit Information System**: registration, penalties, abnormal operations;
- **China Judgements Online / Enforcement Information**: legal and risk information (mind the terms of use).

Research databases (mostly institutional subscriptions)

- **CSMAR**: Chinese economic and financial research database (academic; documented fields; citation requirements);
- **CNRDS**: Chinese research data service platform (text, ESG, patent and other specialty libraries);
- **Wind / iFinD / CEIC**: institutional market and macro databases;
- **CEInet statistical database**: macro and industry statistics.

Check the campus library first: much "expensive data" is free on campus.

Data-as-a-Service (International)

Finance and business

- Bloomberg Terminal / Refinitiv (LSEG): quotes, financials, news, ESG;
- FactSet, S&P Capital IQ: fundamentals and valuation data;
- MSCI / FTSE Russell: index and factor data;
- Traits: standardised fields, traceable history, SLAs; costly, strict licences (often internal use only).

Consumer, market and web data

- Statista, Euromonitor: market and industry statistics;
- Similarweb, Sensor Tower: traffic and app data;
- Nielsen / Kantar: retail and consumer panels;
- Procurement checks: sample definition, coverage, update lag, permission for secondary analysis.

Source: Vendor information from public product documentation (2026); licence scope is set by contract.

Third-party Data Services (China)

Database	Main users	Features and fit
CSMAR	universities / researchers	built for academic research; clean tables with field documentation; suits papers and teaching
Wind	institutional investors	broadest coverage, fastest updates; terminal + API; suits investment research and live analysis
iFinD (Tonghuashun)	institutions / researchers	market and financial data; relatively cheap; friendlier interface
CNRDS	universities / researchers	specialty libraries for text, ESG, patents, hiring; suits text and innovation research
CEIC China database	institutions / researchers	high-frequency macro and industry indicators; long series, fully annotated definitions
CEInet / DRCnet	universities	macro and industry statistics, research reports; suits teaching and reviews

Academic-use essentials: cite the database and version, respect the single-user licence (no account sharing), and never redistribute exported data.

Source: Official database documentation and university library procurement information (extends the original deck's CSMAR pages).

CSMAR vs. Wind: How to Choose?

Dimension	CSMAR	Wind
Positioning	academic research database	institutional investment-research terminal
Core customers	universities, researchers	brokers, funds, banks, firms
Data organisation	tables by research topic, with field and schema documentation	extremely broad asset coverage; real-time and consistency first
Update frequency	mostly daily/monthly/annual; announcements follow	second/minute-level quotes, live news and announcements
Interfaces and tools	data export + some APIs (Python/R/Stata support)	Excel add-in + licensed API + terminal queries
Cost	relatively low (usually campus-wide licence)	high (terminal licence, billed per seat)
Typical uses	empirical papers, teaching cases, panel data	investment research, factor backtests, live monitoring
Caveats	citation rules, version differences	strict licence, data must not leave the organisation

Source: Extends the original "CSMAR vs Wind" slide with interface and licensing points; defer to official documentation and campus subscription scope.

Dataset Platforms

International platforms

- **Kaggle**: datasets + notebooks + competitions (incl. real business data);
- **Hugging Face Datasets**: NLP/multimodal datasets with loading scripts and cards;
- **UCI / cloud-provider open data**: classic teaching benchmarks and large open sets (satellite, genomic, transport).

Chinese platforms

- **Tianchi (Alibaba)**: competitions and industry datasets (e-commerce, finance, logistics);
- **Heywhale**: Chinese-language dataset and notebook sharing;
- **DataFountain / China open-data platform**: government and corporate competition data, local open-data aggregators.

Before using platform data check: the licence (CC/custom), whether commercial use is allowed, whether it contains personal information, and whether redistribution is allowed.

Source: Extends the original Kaggle slide with Chinese platforms; each platform's terms are on its official site.

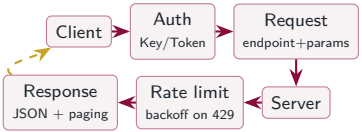
Platform APIs

Category	Representative APIs	Barriers / caveats
Social sentiment	X/Twitter API, Weibo open platform, Reddit	mostly paid tiers; historical data restricted
Maps and location	AMap/Baidu Maps open platforms, Google Maps, OpenStreetMap	free quotas + commercial limits; OSM is ODbL
E-commerce	Amazon SP-API, Taobao open platform, JD Zeus	merchant authorisation needed; price/sales often unavailable
Market data	yfinance, Alpha Vantage, Tushare, AKShare, exchange feeds	free quotas, rate limits, delay, commercial restrictions
Academic literature	OpenAlex, Crossref, Semantic Scholar, arXiv	mostly open; polite pools require an e-mail
Company registries	national credit-information system, Tianyancha/Qichacha (commercial)	anti-scraping is strict; use official or licensed channels

Preference order: official API $\rightarrow$ commercial licence $\rightarrow$ scraping (last resort); an API's "free" usually means quotas, delay or field restrictions.

REST Fundamentals

- **Request**: method (GET/POST), endpoint, parameters, headers;
- **Authentication**: API key (URL/header), bearer token, OAuth 2.0 (refresh tokens);
- **Response**: JSON/XML/CSV; HTTP status codes (200/401/403/404/429/5xx);
- **Pagination**: offset/limit, page/per_page, cursor — three patterns;
- **Rate limits**: QPS/daily quotas, concurrency caps; back off and retry on 429;
- **Caching and idempotency**: cache identical requests to disk; design idempotency keys for writes.



Source: General REST and API-engineering practice; pagination and rate-limit patterns compiled from mainstream data-service documentation.

Code: Generic API Skeleton — Sessions, Retries, Pagination, Caching

```python
import json, hashlib, pathlib, requests  # stdlib (json/hashlib/pathlib) + HTTP library
from requests.adapters import HTTPAdapter # adapter: mounts the retry policy
from urllib3.util.retry import Retry     # urllib3 retry strategy class

CACHE = pathlib.Path("cache"); CACHE.mkdir(exist_ok=True) # local cache dir (reused if present)
def make_session(token=None):            # build a session: auth, retries, timeouts in one place
    s = requests.Session()               # a session reuses TCP connections, saving handshakes
    s.mount("https://", HTTPAdapter(max_retries=Retry( # attach retries to https
        total=5, backoff_factor=0.8,     # up to 5 tries, 0.8 s exponential backoff
        status_forcelist=[429, 500, 502, 503, 504], allowed_methods=["GET"]))) # retry GET only
    s.headers.update({"User-Agent": "szu-course/1.0 (you@szu.edu.cn)", # declare identity and contact
                      "Accept": "application/json"}) # accept JSON responses only
    if token: s.headers["Authorization"] = f"Bearer {token}" # token passed in from the environment
    return s                             # return the configured session

def get_json(session, url, params=None, use_cache=True): # fetch JSON: cache first, then request
    key = hashlib.md5((url + json.dumps(params or {}, sort_keys=True)).encode()).hexdigest() # cache key
    fp = CACHE / f"{key}.json"           # cache file path built from the key
    if use_cache and fp.exists(): return json.loads(fp.read_text(encoding="utf-8")) # hit -> return
    r = session.get(url, params=params, timeout=30); r.raise_for_status() # 30 s timeout, check status
    data = r.json()                      # parse the body into Python objects
    fp.write_text(json.dumps(data, ensure_ascii=False), encoding="utf-8") # persist for next time
    return data                          # hand the data back to the caller
```

Key point: wrap "auth / retries / caching / timeouts" into one layer; business code only cares about endpoints and fields.

Code: International Source Example — Bulk Pull from the World Bank API

```python
import pandas as pd                       # hold the result in a DataFrame
from api_utils import make_session, get_json # session + fetch helpers from the previous slide

BASE, IND = "https://api.worldbank.org/v2", "NY.GDP.MKTP.CD" # endpoint and indicator (GDP, current USD)
s = make_session()                        # session with retries and caching
rows, page = [], 1                        # result list and page counter (start at page 1)
while True:                               # paging loop: until meta.pages is exhausted
    js = get_json(s, f"{BASE}/country/CHN;USA/indicator/{IND}", # GDP for China and the USA
                  {"format": "json", "per_page": 100, "page": page}) # 100 records per page
    meta, data = js[0], js[1]             # World Bank returns [metadata, data]
    rows += [{"country": d["country"]["value"], "year": int(d["date"]), # country and year
              "gdp_usd": d["value"]} for d in data if d["value"] is not None] # drop nulls
    if page >= meta["pages"]:             # last page reached
        break                             # end the paging loop
    page += 1                             # otherwise move to the next page

df = pd.DataFrame(rows).sort_values(["country", "year"]) # tabulate and sort by country, year
print(df.groupby("country")["gdp_usd"].last()) # latest-year GDP
df.to_parquet("wb_gdp.parquet")               # persist + version-comparable
```

Swap the indicator codes (e.g. trade, population, education) and the country list in the same pattern to build panel data in bulk.

Source: World Bank Indicators API v2 (api.worldbank.org/v2) public documentation.

Code: Market Prices and Macro Time Series (International)

```python
import yfinance as yf, pandas as pd      # market-data + tabular libraries

tickers = ["AAPL", "MSFT", "0700.HK", "600519.SS"] # US, HK and A-share representatives
px = yf.download(tickers, start="2020-01-01", end="2025-12-31", # bulk daily prices
                 auto_adjust=True, progress=False, group_by="column") # adjusted, quiet, column-grouped
close = px["Close"]                      # closing prices: rows = dates, cols = tickers
rets = close.pct_change().dropna()       # daily returns, drop the first NaN
print(rets.describe().T[["mean", "std", "min", "max"]]) # mean/volatility/extremes overview

# Macro: FRED (free API key) -- via pandas_datareader or requests
from pandas_datareader import data as web # unified entry point for macro time series
cpi = web.DataReader("CPIAUCSL", "fred", "2015-01-01", "2025-12-01") # US CPI, monthly series

# Note: bulk downloads easily hit rate limits -> batch + pause + local cache (wrapper on slide 35)
```

Teaching tip: for price data always state the adjustment method, time zone and trading calendar, or backtest results cannot be compared.

Source: yfinance and FRED/pandas-datareader official documentation; ticker suffix rules (.SS/.SZ/.HK) per exchange guides.

Code: Chinese Sources — Tushare Pro and AKShare

```python
# ---------- Tushare Pro: register for a token; some endpoints need points ----------
import tushare as ts                    # import Tushare (A-shares and macro)
ts.set_token("YOUR_TOKEN")      # an environment variable is better: os.getenv("TS_TOKEN")
pro = ts.pro_api()                      # initialise the pro API object with the token
daily = pro.daily(ts_code="000001.SZ", start_date="20240101", end_date="20241231") # daily bars
gdp  = pro.cn_gdp(start_q="2015Q1", end_q="2024Q4") # quarterly macro: China GDP
codes = pro.stock_basic(exchange="", list_status="L") # all listed stocks, basic info

# ---------- AKShare: free, no token, wraps public pages/endpoints ----------
import akshare as ak                    # import AKShare (free interface collection)
hist = ak.stock_zh_a_hist(symbol="000001", period="daily", # daily A-share bars
                          start_date="20240101", end_date="20241231", adjust="qfq") # forward-adjusted
cpi  = ak.macro_china_cpi_monthly()                  # monthly CPI
trade = ak.macro_china_trade_balance()               # trade balance
```

Both libraries depend on upstream page structures and their interfaces may change: pin the versions (requirements) and keep on-disk snapshots.

Source: Tushare Pro and AKShare official documentation (2026); data for teaching and research only — commercial use must follow platform terms.

Web Scraping (1): Principles and Tools

- Static pages: parse the HTML directly (requests + BeautifulSoup/lxml);
- dynamic pages: JS-rendered → need Playwright/Selenium (or hidden JSON endpoints);
- common targets: list-page pagination, detail-page fields, HTML tables, attached PDFs;
- engineering: request spacing, retry on failure, UA declaration, resumable crawls, on-disk snapshots.

```python
import requests, pandas as pd # HTTP + tables
from bs4 import BeautifulSoup # HTML parser (lxml backend)

url = "https://example.org/stats?page=1"  # target list page
html = requests.get(url, timeout=20, # fetch page (20 s timeout)
       headers={"User-Agent": "szu-course/1.0"}).text # declare self
soup = BeautifulSoup(html, "lxml") # parse into a DOM tree

# Approach 1: extract fields with CSS selectors
rows = [{"name": tr.select_one("td.name").get_text(strip=True), # name
         "value": tr.select_one("td.val").get_text(strip=True)} # value
        for tr in soup.select("table#data tbody tr")] # all data rows

# Approach 2: read HTML tables directly (tidy tables)
tables = pd.read_html(html)     # returns a list of DataFrames
df = tables[0]                  # take the first table
```

Visual scraping tools mentioned in the original deck (Import.io-style products) suit one-off jobs; for reproducible research prefer code.

Web Scraping (2): Legal and Ethical Boundaries

- Read robots.txt and the site's terms of service first: if scraping is banned, switch sources;
- control the rate (suggest $\geq$1 s/request, concurrency $\leq$2) to avoid harming the service;
- declare your identity and contact clearly (User-Agent) and provide an opt-out;
- never scrape private personal data, paywalled content, or login-restricted data;
- respect copyright and database rights: redistributing text/images needs permission;
- record scrape times and snapshots so research stays reproducible and auditable.

```python
import urllib.robotparser as rp # stdlib: parse robots.txt
UA = "szu-course/1.0"           # crawler identity (with contact)
r = rp.RobotFileParser()        # create the robots parser
r.set_url("https://example.org/robots.txt") # robots.txt
r.read()                        # fetch and parse the rules
print(r.can_fetch(UA, "https://example.org/stats")) # may I?

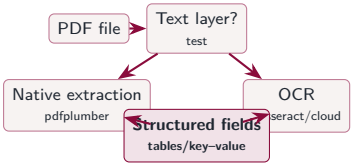
# Polite crawling: spacing + retries + only needed fields
import time, requests           # timing + requests
for page in range(1, 6):        # first 5 pages, limit impact
    resp = requests.get(f"https://example.org/stats?page={page}",
                        headers={"User-Agent": UA}, timeout=20)
    if resp.status_code == 429: # rate-limited
        time.sleep(30); continue # wait 30 s
    save(resp.text, page); time.sleep(1.2) # snapshot; pause
```

Compliance red lines: private data, restricted content, circumventing technical protection measures — never.

PDF, Scanned and Document Data

Two kinds of PDF, handled completely differently

- Native PDFs (selectable text): extract text/tables directly, high accuracy;
- Scanned PDFs (images): need OCR (Tesseract, cloud OCR) before field extraction;
- how to tell: try selecting text / check for font information.



Source: Document-extraction tools (pdfplumber, Camelot, Tesseract) are standard open-source options; scan quality caps OCR accuracy.

Code: PDF Text and Table Extraction

```python
import pdfplumber, pandas as pd          # PDF parsing + table handling

# 1) extract text page by page (good for key-value pairs, paragraphs)
with pdfplumber.open("annual_report_2024.pdf") as pdf: # open the annual report
    text = "\n".join((p.extract_text() or "") for p in pdf.pages[:5]) # join the first 5 pages

# 2) extract tables (good for the three financial statements, statistical annexes)
with pdfplumber.open("annual_report_2024.pdf") as pdf: # open again (tables extracted separately)
    tables = [pd.DataFrame(t[1:], columns=t[0])  # first row as header, rest as data
              for p in pdf.pages for t in p.extract_tables() if t] # every table on every page
fin = tables[0].replace({"": None}).dropna(how="all") # empty strings to NaN, drop empty rows

# 3) scans: OCR first, then extract (needs tesseract + the chi_sim language pack)
# from pdf2image import convert_from_path; import pytesseract # the OCR toolkit
# pages = convert_from_path("scanned.pdf", dpi=300)    # PDF to 300 dpi images
# text = "\n".join(pytesseract.image_to_string(p, lang="chi_sim+eng") for p in pages) # Chinese+English OCR
```

Practical note: financial-statement tables often span pages / contain merged cells; after extraction always run an additivity check (sum of items vs total).

Source: pdfplumber, pytesseract/pdf2image official documentation; Chinese OCR needs the chi_sim language pack.

Code: Table Normalisation and Multi-source Merging

```python
import pandas as pd, re                  # table handling + regex cleaning

# 1) reading: set encoding and dtype explicitly, so leading zeros survive and "1,234" stays numeric
df = pd.read_csv("sales_2024.csv", encoding="utf-8-sig", thousands=",", # handle BOM and thousands separators
                 dtype={"store_id": "string", "sku": "string"})  # code-like fields as strings

# 2) money and percent parsing (raw fields often carry currency symbols and spaces)
df["amount"] = (df["amount_raw"].astype(str)            # to string for cleaning
                .str.replace(r"[^\d.\-]", "", regex=True).astype(float)) # strip non-numeric chars, to float
df["growth"] = df["growth_raw"].str.rstrip("%").astype(float) / 100 # "12%" -> 0.12

# 3) category alignment: unify spellings across sources into standard codes
region_map = {"Shenzhen": "SZ", "SZX": "SZ", "Shenzhen, CN": "SZ", "Guangdong": "GD"} # alias map
df["region"] = df["region_raw"].map(region_map).fillna("UNK") # unmatched -> UNK

# 4) multi-source merging: keep source tags, dedupe on the business key
a = pd.read_csv("src_a.csv").assign(source="A")        # source A, tagged
b = pd.read_csv("src_b.csv").assign(source="B")        # source B, tagged
allx = (pd.concat([a, b], ignore_index=True)           # stack into one long table
        .drop_duplicates(["date", "store_id", "sku"], keep="last")) # dedupe on key, keep latest
```

Principle: normalise first, merge second; be able to explain "why" for every transformation step.

Code: Structured Extraction with an LLM (with Validation and Sampled Review)

```python
import os, json, jsonschema              # env vars, JSON, schema validation
from openai import OpenAI                # any OpenAI-compatible provider works

client = OpenAI(api_key=os.environ["LLM_API_KEY"]) # key read from the environment, never committed
MODEL = os.getenv("LLM_MODEL", "gpt-5-mini") # or switch to a domestic model endpoint

SCHEMA = {"type": "object", "required": ["company", "amount_million", "sign"], # three required fields
        "properties": {"company": {"type": "string"},        # company name: string
                       "amount_million": {"type": ["number", "null"]}, # amount: number or null
                       "sign": {"enum": ["positive", "negative", "neutral"]}}} # stance: enum
SYS = "You are a financial-text extraction engine: output JSON only, use null when unsure, never invent." # system prompt

def extract(text: str) -> dict:          # one text -> structured dict
    r = client.chat.completions.create(  # call the LLM
        model=MODEL, temperature=0,      # temperature 0: as deterministic as possible
        response_format={"type": "json_object"}, # force JSON output
        messages=[{"role": "system", "content": SYS},   # system prompt
                  {"role": "user", "content":                # user prompt
                   f"Extract per schema: {json.dumps(SCHEMA, ensure_ascii=False)}\n\n{text[:4000]}"}]) # truncate long texts
    obj = json.loads(r.choices[0].message.content) # parse the returned JSON string
    jsonschema.validate(obj, SCHEMA)         # schema check; retry/drop if invalid
    return obj                           # return the validated result
# Sampled review: compare 50 random records against human labels and report the agreement rate
```

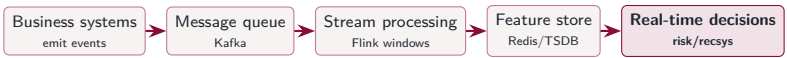
Three necessities for LLM extraction: schema validation + sampled human review + archived prompts.

Streaming & Real-time Data

- **Event streams**: clicks, payments, location, device heartbeats; message queues (Kafka, Pulsar);
- **Stream processing**: Flink/Spark Structured Streaming for windowed aggregation and alerting;
- **CDC**: database change capture (binlog/WAL) $\rightarrow$ real-time sync into the warehouse;
- **Real-time features**: risk control and recommendation need the newest features within milliseconds to seconds.

The batch vs stream triangle

- Latency: batch (hours–days) $\rightarrow$ micro-batch (minutes) $\rightarrow$ stream (seconds);
- Cost: real-time pipelines are significantly more complex to operate;
- Consistency: batch is easier to rerun and audit; streams need idempotency and exactly-once design.



Source: Streaming architecture (Kafka/Flink/CDC) is standard industry practice; teaching projects can usually stay with batch processing.

Alternative Data

Type	Typical signals	Common sources / vendors
Satellite & geospatial	parking-lot traffic, night lights, crop vigour, mining activity	imagery vendors + in-house recognition models
Shipping & logistics	vessel position and speed, port congestion, container flows	AIS data, port and carrier public data
Consumption & payments	card spending, store footfall, delivery orders	card networks / aggregators (mostly licensed)
Hiring & labour	job postings and skill demand, headcount flows	job-board scraping or commercial vendors
Apps & web	app downloads and usage, web traffic, search indices	third-party trackers, public indices (e.g. Baidu Index)
Sentiment & text	news sentiment, social chatter, analyst views	news APIs, social platform APIs, in-house crawlers

Alternative data earns its keep through lead time and non-consensus signal; the risks are coverage bias, sample changes, compliance and sustainability.

Source: The alternative-data taxonomy is compiled from industry surveys and public product documentation; actual availability depends on licensing and region.

Alternative Data: Evaluation and Compliance Checklist

Technical and statistical evaluation

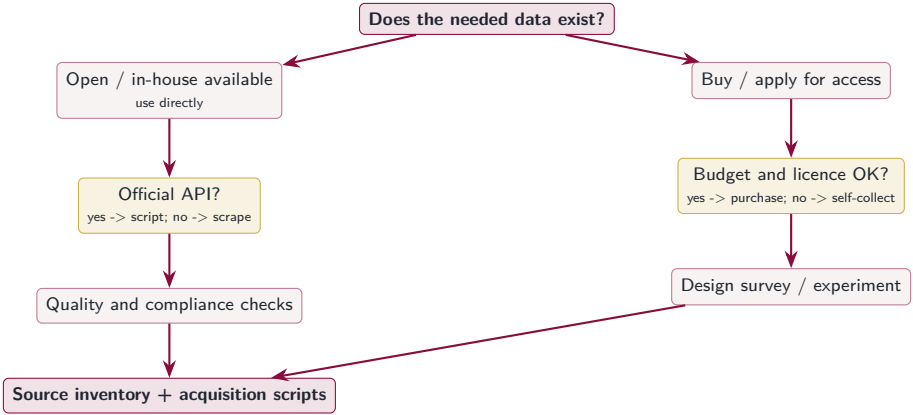
- Coverage: how much real activity does the sample cover? (sampling bias)
- Signal strength: correlation with the target and lead time;
- Stability: vendor definition/sample changes break historical comparability;
- Explainability: can you say why the signal works?;
- Cost-benefit: data fee + cleaning + modelling worth it?

Compliance and ethics

- Does it contain personal information? Is it licensed? Allowed for this purpose?
- Does it touch privacy (location traces, spending detail)?
- National security / cross-border transfer issues?
- Is the vendor's own sourcing legal and traceable?
- Could the conclusions harm the observed group?

Ask "may we use it?" before asking "is it useful?"

Source Selection Decision Tree and Summary



Order of preference: official API → commercial licence → scraping → self-collection; the further back, the higher the cost and risk.

Never start with scraping: it is the fallback with the highest maintenance and legal cost.

When Must You Collect Data Yourself?

Three situations

- No secondary data: new products, new markets, internal process improvement;
- Causal evidence needed: only randomised experiments (A/B tests) identify the effect;
- New variables needed: existing data lacks the key constructs (trust, intention, experience).

Count three costs first

- Time: design + pretesting + collection + cleaning, usually 2–6 weeks;
- Money: respondent incentives, platform fees, labour;
- Quality: response rate, sample representativeness, measurement validity.

The new-data sources emphasised in the original deck (interviews, questionnaires, lab experiments, A/B tests) map exactly onto these three situations.

Questionnaire Design (1): Questions and Scales

- **Open questions**: rich but costly to code; **closed questions**: easy to analyse but constrain answers;
- **Likert scales**: 5 vs 7 points; label the anchor meanings; avoid dropping the neutral option;
- **Construct operationalisation**: measure one construct with 3–5 items (multi-item raises reliability);
- **Reverse-coded items**: catch careless responding, but use few (they confuse respondents).

Four tests of a good question

- Single focus (avoid double-barrelled items: "Is the product cheap and good?");
- plain language (no jargon or acronyms);
- no presupposed answer (avoid leading wording);
- options mutually exclusive and exhaustive (include "other / don't know").

Source: General survey-methodology practice (e.g. Tourangeau et al. on questionnaire design and the cognitive process of answering).

Questionnaire Design (2): Biases and Order Effects

Bias	How it shows up	Mitigation
Social desirability	tendency to give "respectable" answers	anonymity, neutral wording, indirect questions
Recall bias	cannot recall past behaviour (frequency/timing)	shorter recall window, anchor events, diary methods
Anchoring	influenced by the previous question or number	shuffle sensitive items, randomise option order
Default / order	picking the first or default option	randomise options, avoid preset defaults
Central tendency	always the midpoint	forced choice, more items
Nonresponse	certain groups don't answer (systematic missing)	reminders, weighting, substitute samples

Most of these biases cannot be "fixed after the fact": they must be handled at the questionnaire-design stage.

Source: Common response-bias taxonomy and mitigation strategies from survey methodology (2020s reviews and practice guides).

Questionnaire Design (3): Pretesting, Length and Dropout

Three-step pretest

1. Expert review: construct coverage, wording, skip logic;
2. Cognitive interviews: 5–10 respondents think aloud while answering, exposing ambiguity;
3. Pilot run: 30–50 responses; check reliability (Cronbach's α), discrimination and completion time.

Length and dropout

- Once the median online completion time passes 10 minutes, dropout rises sharply;
- put key items in the first third, demographics last;
- show a progress bar; allow resuming (save drafts);
- log abandonment points; check whether dropout tracks specific items (systematic).

Time invested in pretesting is repaid several times over at the cleaning stage.

Sampling Methods

Method	How it works	Fit and cost
Simple random	equal-probability draws from the frame	needs a full frame; lowest variance, high cost
Stratified	stratify by key traits, sample each stratum	better representativeness (industry/region/-size)
Cluster	sample clusters (schools/stores/communities), survey all within	cheap; intra-cluster correlation cuts effective sample size
Systematic	every k-th unit	simple; beware periodicity
Convenience / quota	recruit nearby or to quotas	fast and cheap, but cannot infer to the population
Snowball	respondents refer respondents	suits hard-to-reach groups (experts, tiny populations)

Classroom research usually uses convenience/quota samples: declare clearly that the sample cannot infer to the population, and write the limitation into the report.

Survey Modes and the Cost–Coverage Trade-off

Mode	Coverage	Cost	Best for
Face-to-face	broad; can show material, can probe	highest, slow	complex surveys, sensitive topics, elderly
Telephone	landline and mobile; falling response rates	medium-high, call-centre bound	short surveys, quick screening
Mail	stable-address people; low return rates	medium, long cycle	people who avoid phone contact
Online	internet users (skews young)	lowest, fastest	general surveys, experiments, panel tracking
Mixed mode	balances coverage and cost	medium; handle mode effects	large surveys (e.g. official statistics)

The original deck's conclusion still holds: mode choice is constrained by cost and coverage of the target population; use mixed mode to plug single-mode gaps.

Source: The cost–coverage trade-off of survey modes follows standard survey-methodology conclusions; in the Chinese context also consider real-name mobile registration and SMS reach.

Nonresponse & Weighting

- Unit nonresponse: sampled but didn't answer; item nonresponse: answered but left some items blank;
- the danger is non-randomness: high-income, young male and similar groups go systematically missing;
- diagnostics: compare sample vs population age/region/size structure; compare early vs late responders;
- mitigation: reminders and incentives, shorter surveys, phone follow-up, post-hoc weighting.

Weighting and correction

- Post-stratification weighting: weight sample units to the population distribution;
- Inverse-probability weighting: model the "response probability" and use its inverse as the weight;
- caveat: weighting inflates variance — report the effective sample size;
- unfixable case: groups entirely missing from the sample (no overlap).

Source: Nonresponse bias and weighting follow standard survey-statistics practice; directly related to selection problems in L12 causal inference.

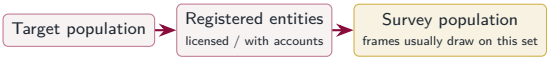
Census & Administrative Data

Census

- National population census (every 10 years), economic census, agricultural census;
- strengths: full coverage, finest geographic detail;
- weaknesses: long cycle, few items, microdata require application.

Administrative records

- Business-system records: registration, tax, social insurance, customs, health insurance;
- strengths: continuous, cheap, covers real behaviour;
- weaknesses: policy-driven definitions; registered entities only.



Source: China's censuses are organised by the NBS; administrative data use follows approval processes.

Experimental Design

- Randomisation: averages out confounders (observed and unobserved);
- Control group: no treatment / placebo / current practice;
- Blinding: avoids expectation effects (subjects and experimenters);
- Power analysis: given effect size and variance, compute the required sample size;
- Preregistration: write hypotheses and primary outcomes first — no dredging afterwards.

Minimum-sample intuition

$$n \approx \frac{16\,\sigma^2}{\Delta^2} \quad \text{(a rough two-group approximation)}$$

where Δ is the effect you want to detect and σ the standard deviation of the outcome. Smaller effects and noisier outcomes need more samples (quadratically).

Source: Standard experimental-design and power-analysis statements; for exact sample sizes use power tools (e.g. G*Power, statsmodels).

A/B Testing (1): From Hypothesis to Interpretation



Three things when reading the result

- Effect size and confidence interval (not just p-values);
- practical significance reached? (cost, experience, long-run impact);
- are segment results consistent? (heterogeneous treatment effects, see L12).

Examples from the original deck (still valid)

- E-mail marketing: subject-line A/B → open and conversion rates;
- product pricing: conversion and revenue at different price points;
- pages / recommendation slots: click-through and dwell time.

A/B Testing (2): Traps and Protections

Trap	Consequence	Protection
SRM (sample-ratio mismatch)	odd split ratios invalidate conclusions	verify the split daily; halt on anomaly
Multiple comparisons	many metrics/segments guarantee a "significant" hit	pre-specify the primary metric; FDR/Bonferroni correction
Early stopping / peeking	repeated peeking inflates false positives	fixed sample size or sequential-test design
Network effects / spillover	control contaminated by treatment (social, shared inventory)	cluster randomisation
Proxy drift	optimising clicks harms long-run value	add guardrail metrics (retention, refunds, complaints)
Seasonality and external shocks	collision with sales events, holidays, policy	concurrent control + log external events

The original deck's "e-mail marketing / pricing A/B" examples become a complete methodology only once this trap list is attached.

Source: A/B-testing engineering practice (SRM detection, sequential tests, cluster randomisation) compiled from industry platforms and the statistics literature.

Lab & Behavioural Experiments

- **Lab experiments**: present stimuli and record responses in a controlled setting (original deck: in-class projects on sharing information or experiences);
- **Measures**: response times, choices and bids, eye tracking, skin conductance/heart rate, facial expression;
- **Experimental-economics paradigms**: public-goods games, ultimatum, trust game, auctions;
- **Strengths**: high internal validity, mechanisms isolated; **weaknesses**: small samples, external validity in doubt (the WEIRD-sample problem).

Online behavioural experiments

- Crowd-platform recruitment + online tasks (mouse trajectories, randomised options);
- combine with surveys: behaviour first, self-report after — less talk-action gap;
- preregistration and informed consent (IRB ethics review).

Source: Experimental-economics and behavioural methods (including the WEIRD-sample critique) are standard academic consensus.

Interviews & Qualitative Methods

- **Semi-structured interviews**: fixed guide, flexible probes; suits new contexts (original-deck view);
- **Focus groups**: group interaction sparks views, but conformity effects must be controlled;
- **Sampling**: purposive, maximum-variation, theoretical saturation (stop when no new themes appear);
- **Cost**: 45–90 minutes per session + transcription + coding; samples of 10–30 are common.

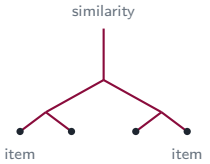
Analysis workflow (thematic analysis)

1. transcribe and immerse in the material;
2. open coding $\rightarrow$ categories $\rightarrow$ themes;
3. double coding and disagreement resolution (report agreement);
4. triangulation: cross-check against surveys, logs and documents.

Qualitative material is for "explaining mechanisms", not for "estimating proportions".

Card Sorting and Structure Discovery

- **Open card sorting**: participants group and name the cards themselves → reveals users' mental models;
- **Closed card sorting**: given categories, assign each card;
- **Output**: similarity matrix (pairwise co-grouping counts) → hierarchical clustering dendrogram;
- **Uses**: information architecture, grouping questionnaire items, validating concept structures;
- both the original text and practice stress: 15–30 participants give stable grouping structures.



Source: Card sorting and dendrograms follow the original deck; clustering distance can be Jaccard or Euclidean.

Crowdsourcing & Agreement

- **Labelling guide**: state decision rules and edge cases (with counter-example screenshots/text);
- **Gold questions**: mix in items with known answers to filter careless labellers;
- **Multiple labels**: 2–3 labellers on key samples; disputes go to arbitration;
- **Agreement metrics**: Cohen's κ / Fleiss' κ / Krippendorff's α;
- **Cost control**: pilot-label a small batch, calibrate the guide, then scale up.

Reading agreement

- $\kappa < 0.4$: poor — rewrite the guide;
- $0.4 \leq \kappa < 0.6$: fair — usable for exploration;
- $0.6 \leq \kappa < 0.8$: good — usable for modelling;
- $\kappa \geq 0.8$: excellent (often required in medical labelling).

Note: with extremely unbalanced classes κ is depressed — report raw agreement alongside.

Source: Standard agreement metrics and empirical thresholds (Cohen 1960; Landis & Koch 1977; Krippendorff).

LLM-assisted Data Collection (2026)

Where it works

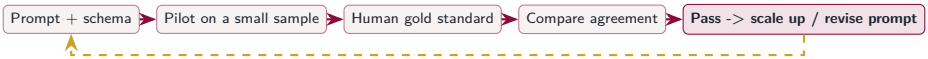
- Text structuring: extract companies, amounts, events, sentiment from announcements/news;
- Zero-shot / few-shot labelling: classification, topics, information extraction;
- Survey and interview support: draft items, tidy transcripts, summarise themes;
- Synthetic data: augment rare classes, generate test cases, simulate respondents.

Where it cannot be relied on

- Fact-precise numbers (it fabricates);
- conclusions needing traceable sources (no citation = not trustworthy);
- direct judgements in high-stakes decisions (credit, medical, legal);
- direct upload of personal-privacy data (compliance risk).

Positioning: the LLM is an efficient first-pass labeller and extractor, not a "source of truth". Humans own the rules, the verification and the accountability.

LLM Assistance: Risks and Verification Workflow



Three risks

- Self-consistency bias: the model leans toward its own priors; errors are not random;
- Drift: after a vendor upgrades the model, the same prompt gives different results;
- Licence and privacy: inputs may be used for training/retention (read the terms).

Must-do actions

- Record the model name, version, date, temperature and the full prompt;
- sample for human review; report agreement rates and error types;
- keep raw text and model outputs; state in the report which fields were model-generated and how they were verified.

Synthetic Data and Privacy-Enhancing Technologies

Synthetic Data

- Uses: augment rare classes, build test sets, protect privacy when sharing;
- methods: rule-based generation, SMOTE-type oversampling, generative models (VAE/GAN/diffusion), LLM generation;
- risks: distribution distortion, amplified bias, "contaminating" later training;
- discipline: synthetic data must be cross-validated against real data, and its provenance labelled separately.

Privacy-enhancing technologies (PETs)

- De-identification/anonymisation: remove or generalise identifiers (watch re-identification risk);
- Differential privacy: noise makes individuals indistinguishable, at a precision cost;
- Federated learning: data never leaves its domain; only model updates are exchanged;
- Secure MPC / homomorphic encryption: joint statistics across institutions without sharing raw data.

Source: Synthetic-data and PET surveys (2020s); the privacy-budget vs accuracy trade-off of differential privacy must be explicit in the design.

Code: Data Health Check — Missing, Duplicates, Outliers, Time Coverage

```python
import pandas as pd, numpy as np          # table handling + numerics

df = pd.read_parquet("data/raw/snapshot_2026-09-01.parquet") # load the raw snapshot
print(f"shape={df.shape} memory={df.memory_usage(deep=True).sum()/1e6:.1f} MB") # size and memory

# 1) missingness: sort by column; watch for "block missing" (whole field groups missing together)
miss = (df.isna().mean() * 100).round(2).sort_values(ascending=False) # missing % per column, descending
print(miss[miss > 0].head(10))            # only the top 10 columns with missing values

# 2) duplicates: full-row duplicates + business-key duplicates
print("dup rows:", int(df.duplicated().sum()))           # fully duplicated rows
print("dup keys:", int(df.duplicated(["date", "store_id", "sku"]).sum())) # duplicated business keys

# 3) outliers: IQR rule gives each numeric column's outlier share (share first, treatment later)
num = df.select_dtypes("number")          # numeric columns only
q1, q3 = num.quantile(.25), num.quantile(.75); iqr = q3 - q1 # quartiles and IQR
outlier = ((num < q1 - 1.5*iqr) | (num > q3 + 1.5*iqr)).mean().round(4) # outlier share
print(outlier.sort_values(ascending=False).head())        # top offenders by share

# 4) time coverage: span, frequency and gaps
d = pd.to_datetime(df["date"])            # unify into a date type
print("range:", d.min().date(), "->", d.max().date())     # first and last dates
print(d.dt.to_period("M").value_counts().sort_index().tail(6)) # record counts of the last 6 months
```

The health-check report is a core deliverable of Assignment 1: write conclusions as "finding + impact + treatment plan".

Summary: Primary-Data Quality-Control Checklist

1. Are constructs measured with multiple items? (reliability)
2. Whom does the sampling frame cover, whom does it miss?
3. Were sample size and power computed in advance?
4. Was the questionnaire pretested and cognitively interviewed?
5. Are social-desirability / order / default biases present?
6. Is nonresponse systematic? Was weighting applied?
7. Is the experiment randomised, with guardrail metrics?
8. Is labelling multi-rater, with agreement reported?
9. Was LLM assistance sample-audited and archived?
10. Are data and materials reproducible, auditable, deletable?

The money in primary data goes into "design and verification", not into "collection".

From Sources to Analysis Tables: ETL vs. ELT



Classic ETL: transform before load. Modern ELT: load first, transform in-warehouse with SQL

ETL suits

strict compliance needs, masking before load, expensive target stores (only processed data is kept).

ELT suits

elastic cloud warehouses, keeping raw detail, frequently changing transformation logic.

Both require

idempotent reruns, resumable failures, change logs, traceable sources (lineage).

Code: A Minimal Viable Pipeline — Snapshot -> Validate -> Analysis Table

```python
import pathlib, logging, datetime as dt  # paths, logging, dates
import pandas as pd                      # table handling

logging.basicConfig(level="INFO", format="%(asctime)s %(levelname)s %(message)s") # unified log format
RAW, CUR = pathlib.Path("data/raw"), pathlib.Path("data/curated") # raw and curated layers
RAW.mkdir(parents=True, exist_ok=True); CUR.mkdir(parents=True, exist_ok=True) # create dirs if missing

def extract() -> pd.DataFrame:          # extract: persist an immutable raw snapshot first
    df = pd.read_csv("https://example.org/api/export.csv") # fetch from the endpoint
    df.to_parquet(RAW / f"snapshot_{dt.date.today().isoformat()}.parquet") # date-stamped snapshot
    logging.info("snapshot rows=%d", len(df)); return df   # log and return

def validate(df: pd.DataFrame) -> pd.DataFrame: # validate: abort on failure
    assert df["id"].is_unique, "id must be unique"              # primary key unique
    assert df["amount"].notna().mean() > 0.99, "too many missing amounts" # missing-rate cap
    assert (df["amount"] >= 0).mean() > 0.99, "negative amounts found" # business rule: non-negative
    return df.assign(date=pd.to_datetime(df["date"]).dt.date) # unify dates, then return

def load(df: pd.DataFrame) -> None:     # load: idempotent write + logs
    df.drop_duplicates(["date", "store_id", "sku"]).to_parquet(CUR / "sales.parquet") # dedupe, persist
    logging.info("curated rows=%d cols=%d", *df.shape)     # record the final size

if __name__ == "__main__":              # when run directly as a script
    load(validate(extract()))           # extract -> validate -> load
```

Use this slide as the script skeleton for Assignment 1: snapshot + assertions + idempotent writes + logging.

Data Contracts & Data Products

Data contract

- Structure: field names, types, units, nullability;
- Semantics: definitions, calculation rules, effective dates;
- Quality: completeness / uniqueness / range constraints with thresholds;
- Service: update frequency, latency promises (SLA), change-notification process.

Data product

- Manage a dataset as a "product": owner, version, documentation, users;
- changes go through review: breaking changes announced ahead of time (deprecation window);
- user-feedback loop: field requests, quality issues logged and fixed.

The value of contracts: upstream changes become predictable, instead of finding all your dashboards empty one morning.

Source: Data-contract and data-product practice (2020s data-engineering consensus).

The Modern Data Stack and Open Table Formats

- **Lakehouse**: object storage + table format + SQL engine; balances cost and transactions;
- **Open table formats**: Iceberg / Delta Lake / Hudi — ACID, time travel, schema evolution;
- **Query engines**: Spark, Trino, DuckDB (local single-node analytics), Polars (in-memory columnar);
- **Transformation layer**: dbt (SQL models + tests + docs), Airflow/Dagster (scheduling and dependencies).

What this means for course projects

- Small data can be engineered too: DuckDB + Parquet + dbt gives a reproducible pipeline;
- keep raw snapshots and derived tables separate, for reruns and audits;
- echoes L1's "modern data stack"; this lecture is about landing it concretely.

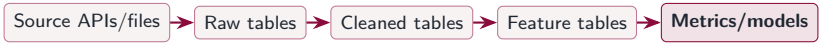
Source: Apache Iceberg, Delta Lake, Hudi, DuckDB, dbt official documentation (2026).

Metadata & Lineage

- **Technical metadata**: tables / fields / types / partitions / file sizes;
- **Business metadata**: metric definitions, rules, owners, update frequency;
- **Lineage**: full-chain dependencies from raw sources to final metrics;
- **Data catalogue**: a searchable asset inventory (who has what data, who may use it).

"Where does this number come from?"

- Good metadata lets any metric be traced back layer by layer to its source;
- when a report misbehaves, the lineage graph pinpoints the upstream change;
- numbers in papers and reports should point to specific tables and versions.



Versions, Snapshots & Reproducibility

- **Code**: scripts under Git; prompts and configs version-controlled too;
- **Data**: raw snapshots immutable (named by date/version); derived tables rebuildable;
- **Environment**: pinned dependency versions (requirements/lock), container images;
- **Randomness**: fix random seeds; record library and hardware versions;
- **Tools**: DVC / Git-LFS for large data; MLflow-style tools to track experiments.

"Reproducible" at assignment/paper level

- Given your repository, anyone can rerun one command and get the same results;
- key numbers in the report map to specific scripts and data versions;
- data sources, acquisition dates and processing steps fully traceable.

Reproducibility is not "extra work"; it is one of the acceptance criteria of the sourcing step.

Data Quality Rules

Dimension	Example rule	Action when triggered
Completeness	key-field non-null rate $\geq 99\%$	block the load + notify the owner
Uniqueness	unique primary keys, no duplicate snapshots	dedupe or halt the pipeline
Range	amounts ≥ 0, shares $\in [0, 1]$	flag and quarantine records
Consistency	items sum to totals; cross-table reconciliation	generate a difference report, human sign-off
Timeliness	data arrives by the agreed time	alert + roll back to the last version
Distribution stability	mean/quantile drift beyond threshold	inspect upstream changes or retrain

Implementation: write the rules as code (Great Expectations / dbt tests / hand-rolled assertions) and run them on every execution and in CI.

Source: Data-quality dimensions and rules-as-code practice (Great Expectations, dbt tests documentation).

Privacy and Compliance: GDPR/CCPA and PIPL

Dimension	GDPR (EU) / CCPA (California)	China PIPL / Data Security Law
Lawful basis	six bases incl. consent, contract, legitimate interest	consent, contract necessity, legal duty, etc.
Notice and consent	clear notice, withdrawable	separate consent (sensitive data), notification duties
Individual rights	access, correction, deletion, portability, objection	access, copying, correction, deletion (conditions apply)
Cross-border transfer	adequacy decisions, SCCs and similar mechanisms	security assessment, standard contract, certification
High-risk processing	DPIA (data protection impact assessment)	personal-information protection impact assessment
Penalties	up to 4% of global turnover	fines, business suspension, penalties for responsible persons

At the course-project level: collect no personal information you do not need, anonymise wherever possible, and read the terms and transfer requirements even for public data.

Source: GDPR (EU 2016/679), CCPA/CPRA, China's PIPL and Data Security Law, and related cross-border transfer rules.

Copyright, Licences, Security and Ethics

Copyright and licences

- "Publicly accessible" $\neq$ "freely usable";
- common licences: CC BY / CC BY-NC / ODbL (open database licence) / custom commercial terms;
- database rights: the EU has sui generis database rights;
- papers and reports: cite the source, version, access date and licence.

Security and ethics practice

- Least privilege, access on need, revoke on departure;
- encrypt in transit and at rest; mask/encrypt sensitive fields;
- access audit logs (who saw what, when);
- protect vulnerable groups: extra limits on children's, patients' and employees' data;
- transparency: data subjects know the purpose and their rights.

Compliance is not a final checkpoint; it is the starting point of sourcing design.

Translating Research Questions into Data Needs

Research question	Key constructs	Variables / definitions needed	Candidate sources
How does digital trade affect SME exports?	digital-trade intensity, export performance	firm exports (customs basis), e-commerce platform use, regional digitalisation index	customs statistics, firm surveys, local open data
Does ESG performance lower financing costs?	ESG scores, financing costs	bond spreads, ratings, ESG scores (definition and vintage)	CSMAR/CNRDS, Wind, bond disclosures
Can sentiment predict short-run stock volatility?	sentiment, volatility	news/analyst-report text, intraday returns, trading volume	news APIs, cninfo announcements, market data feeds
Consumer response to pricing strategy	price sensitivity, purchase intention	experiment arms, conversion rate, basket size	own A/B experiments, platform back-end data

How to write it: state the "construct" first, then the "observable proxy", and only then the "data source" — in the reverse order the data ends up driving you.

Data-source Inventory Template and Failure Troubleshooting

Data-source inventory (one row per source)

- **Source name**: institution / URL / contact (maintainer, who answers);
- **Content and fields**: key fields, units, definitions;
- **Time coverage**: span, frequency, publication lag;
- **Access**: API / download / scraping (auth needed?);
- **Cost, licence, quality**: price, licence, redistribution; coverage/quality/cost/compliance/reproducibility scores (1–5).

Acquisition-failure checklist

- 401/403: expired key, insufficient rights, missing User-Agent;
- 429: rate-limited $\rightarrow$ exponential backoff + cache;
- schema change: renamed/nested fields $\rightarrow$ add schema assertions;
- encoding mojibake: UTF-8/GBK $\rightarrow$ set encoding explicitly;
- missing data: unfinished pagination, calendar mismatch;
- silent failure: empty results read as "no data" $\rightarrow$ log codes.

Assignment 1: Data Sourcing and Preparation

Deliverables (individual work)

1. **Dataset**: public or licensable data of your choice ($\sim$10k–100k rows, at least 2 field types);
2. **Data-source inventory**: per the template on slide 89, with licence and access mode;
3. **Acquisition script**: re-runnable (cache, retries, logging) + dependency list;
4. **Health-check report**: missing/duplicate/outlier/time coverage + treatment plan;
5. **AI-use declaration**: tools, versions, key prompts, verification method.

Grading (100 points)

- Legal and reproducible sourcing (25);
- acquisition-code quality and robustness (25);
- health-check depth and issues found (25);
- report clarity and conventions, compliant AI declaration (25).

Submit: repository link + PDF report ($\leq$8 pages); before class in Week 4.

Source: Corresponds to "Assignment 1 due" in Week 4 of the syllabus; project requirements (choose 1 of 4) announced separately.

Common Pitfalls

1. Scraping just because you can, ignoring robots/ToS;
2. feeding category codes into arithmetic as numbers;
3. mixing the same metric under different definitions (e.g. "exports" from different sources);
4. merging two sources while ignoring units and currency;
5. drawing population conclusions from a non-inferable sample;
6. scaling a survey without a pretest, forcing mid-flight changes;
7. peeking repeatedly at A/B results and stopping early;
8. labelling without agreement checks — error taken as signal;
9. letting an LLM write conclusions with no sampled review;
10. no snapshots, no versions — unreproducible three months later.

Mistakes made at the sourcing stage can barely be repaired by later modelling.

References

Books

- McKinney, W. (2018). *Python for Data Analysis* (Ch. 6 data loading and storage);
- Provost & Fawcett (2013). *Data Science for Business* (acquisition and preparation);
- Groves et al. (2009). *Survey Methodology* (questionnaires and nonresponse);
- Kohavi, Tang & Xu (2020). *Trustworthy Online Controlled Experiments*;
- Tourangeau et al. (2000). *The Psychology of Survey Response*.

Documentation and standards

- World Bank API, FRED, OpenAlex, Crossref official documentation;
- Tushare / AKShare documentation and field guides;
- Great Expectations, dbt, Apache Iceberg documentation;
- GDPR, CCPA/CPRA, China's PIPL and Data Security Law;
- Kaggle / Hugging Face dataset-card conventions.

Source: Full links in the course cloud drive, folder "L2 reading and data sources".

Data-source Cheat Sheet (International)

Name	Content	Access
World Bank	development indicators (200+ countries, decades)	api.worldbank.org/v2 (JSON, free)
IMF / OECD	financial statistics, WEO, structural indicators	data portals + SDMX APIs
UN Comtrade	bilateral trade (HS codes)	web + API (limited quota)
Eurostat	EU statistics	JSON-stat API
FRED	US and global macro series	REST API (key required)
OpenAlex / Crossref	publication and citation metadata	open REST API (include an e-mail to be polite)
SEC EDGAR	US-listed filings and financial data	data.sec.gov (User-Agent declaration required)
Kaggle / HF Datasets	datasets and benchmarks	web download / API / `datasets` library
Common Crawl	monthly web snapshots	S3/HTTP bulk download (self-hosted parsing)

Source: Official documentation of each platform (2026); commercial sources (Bloomberg/Refinitiv/FactSet/Statista) on slide 39.

Data-source Cheat Sheet (China) and Key Terms

Name	Content / access
National Bureau of Statistics	macro and census data (web queries + some interfaces)
People's Bank of China	aggregate financing, credit, rates (Excel/PDF tables)
General Administration of Customs	import/export by commodity and country (online query)
SSE / SZSE	quotes, announcements, margin data (sites + licensed feeds)
cninfo	listed-company announcements and annual-report PDFs
Enterprise credit-info system	registration and penalty information (national enterprise credit information system)
CSMAR / CNRDS	academic research databases (campus subscription)
Wind / iFinD / CEIC	institutional data terminals and APIs
Tushare / AKShare	Python data interfaces (free / points-based)
Tianchi / Heywhale / DataFountain	Chinese competitions and open datasets

Key terminology (Chinese–English)

- data source; data contract
- data lineage; snapshot
- de-identification; differential privacy
- federated learning; alternative data
- synthetic data; nonresponse bias
- inter-rater agreement; rate limiting



Thanks!

Q&A · Data Analytics Module M2 · Data Sourcing

Next: Lecture 3 — Data Preparation (merging, cleaning, missing values and outliers)

Xieer Dai (David) · xieer.dai@szu.edu.cn · Sifang Building South 207